

Trick No. 2 (2D $\rightarrow$ 1D Compression)

Problem Reference

D - Count Subgrid Sum = K

What is 2D $\rightarrow$ 1D Compression?

Many rectangle problems appear to require searching over four boundaries: top, bottom, left and right. A brute-force solution enumerates all possible rectangles, requiring

$$O(H^2W^2),$$

which is too slow for large grids.

The key observation is that if we first fix the top and bottom rows, then the height of the rectangle becomes fixed. The only remaining decisions are the left and right columns. Thus the two-dimensional rectangle problem is transformed into a one-dimensional interval problem.

Compression Step

Suppose the top row is r_1 and the bottom row is r_2 .

Construct an array A where

$$A_j = \text{sum of all values in column } j \text{ between } r_1 \text{ and } r_2.$$

Every rectangle between the fixed rows now corresponds to exactly one contiguous subarray of A .

Instead of searching rectangles, we only search subarrays.

Prefix Sum Transformation

Construct the prefix sum array

$$B_j = A_1 + A_2 + \cdots + A_j,$$

with

$$B_0 = 0.$$

The sum of columns $(l + 1, \dots, r)$ is

$$B_r - B_l.$$

Therefore the rectangle problem becomes finding pairs (l, r) satisfying

$$B_r - B_l = K.$$

Editorial Observation

Instead of directly counting pairs with difference exactly K , count

$$B_r - B_l \geq K$$

and

$$B_r - B_l > K.$$

Their difference equals the number of pairs satisfying

$$B_r - B_l = K.$$

This transformation is often useful because inequalities are easier to count than exact values.

Why Sliding Window Works

Every cell contains either 0 or 1.

Therefore every compressed value satisfies

$$A_j \geq 0.$$

Hence the prefix sums never decrease.

Because the prefix sum array is monotonic, while moving the left pointer forward, the smallest valid right pointer never moves backward.

Therefore both pointers move only from left to right.

This is exactly the condition required for the Sliding Window (Two Pointers) technique.

Thus each compressed array can be processed in

$$O(W).$$

Overall Complexity

There are

$$O(H^2)$$

choices for the pair of rows.

Each pair requires

- building the compressed array,
- processing it with sliding window.

Each costs $O(W)$.

Hence the final complexity becomes

$$O(H^2W).$$

Recognition Pattern

Think about this technique whenever

1. the problem involves rectangular regions,
2. the rectangle property depends only on its contents (sum, frequency, etc.),
3. brute force is $O(H^2W^2)$,
4. fixing two boundaries leaves only one-dimensional intervals.

After compression, solve the resulting 1D problem using the most suitable algorithm such as

- Prefix Sum + Hash Map,
- Sliding Window,
- Binary Search,
- Segment Tree,
- Fenwick Tree.

Notice that the compression step remains the same; only the one-dimensional algorithm changes.

General Mental Process

1. Fix two rows (or two columns).
2. Compress the matrix into a 1D array.
3. Express every rectangle as a subarray.
4. Ask: “Which 1D algorithm solves this subarray problem?”
5. Combine the answer for every pair of fixed rows.

One-Sentence Intuition

Fix two boundaries so that a rectangle becomes a subarray; then solve the easier one-dimensional problem.

Practice Problems

1. AtCoder rectangle-sum style problems.
2. CSES – Forest Queries (2D Prefix Sum).
3. LeetCode 1074 – Number of Submatrices That Sum to Target (Hash Map version).
4. Variants where the compressed array is solved using Sliding Window or Prefix Sum.